

CUBICLE NO 8

RAGAVARSHINI A (23AIB12)

SANJAY M (23AIB23)

SANJEEVIKUMAR M (23AIB27)

SINDHUJASREE R (23AIB33)

SIVASAKTHI R(23AIB36)

SOWMIYA G(23ECB38)

SOWMIYA P(23ECB39)

Day 2: Arduino Uno Internal LED Blinking

- **Date:** 22-07-2025
- **Domain:** Embedded Systems and IoT
- **Topic:** Internal LED Blinking using Arduino Uno

Objective

To blink the internal LED on the Arduino Uno and display its ON/OFF status on the Serial Monitor. Further, modify the program to:

- Keep the LED always ON.
- Control the LED state dynamically using Serial Monitor commands.

Components Used

- Arduino Uno Board
- USB Cable
- Arduino IDE (software)

Theory

- The Arduino Uno has a built-in LED connected to digital Pin 13.
- `Serial.begin()` initializes communication with the Serial Monitor.
- `Serial.print()` and `Serial.println()` are used for displaying messages.
- The `delay()` function creates time gaps between LED state changes.
- Using `digitalWrite()` with HIGH keeps the LED ON continuously.
- Serial input allows interaction with the microcontroller, enabling real-time control.

Procedure

1. Connected Arduino Uno board to PC via USB.
2. Opened Arduino IDE, selected Board: Arduino Uno and the correct COM port.
3. Wrote a program to blink the internal LED and print status to the Serial Monitor.
4. Uploaded the code and observed the blinking and messages on the Serial Monitor.
5. Modified the code to keep the LED ON continuously.
6. Further extended the code to accept user input ("ON" or "OFF") from Serial Monitor to control the LED.

Arduino Code

1. Blink & Monitor LED

```
void setup() {  
    pinMode(13, OUTPUT);    // Set pin 13 as output  
    Serial.begin(9600);     // Start Serial communication  
}
```

```
void loop() {  
    digitalWrite(13, HIGH); // LED ON  
    Serial.println("LED is ON");  
    delay(1000);            // Wait 1 second  
  
    digitalWrite(13, LOW);  // LED OFF  
    Serial.println("LED is OFF");  
    delay(1000);            // Wait 1 second  
}
```

2. Always ON Version

```
void setup() {  
  pinMode(13, OUTPUT);  
  digitalWrite(13, HIGH);    // Turn ON LED permanently  
  Serial.begin(9600);  
  Serial.println("LED is always ON");  
}
```

```
void loop() {  
  // Nothing here  
}
```

3. Serial Monitor Controlled LED

String input;

```
void setup() {  
  pinMode(13, OUTPUT);  
  Serial.begin(9600);  
  Serial.println("Type ON or OFF to control the internal LED");  
}
```

```
void loop() {  
  if (Serial.available()) {  
    input = Serial.readStringUntil('\n'); // Read input until Enter key  
    input.trim(); // Remove extra spaces/newlines
```

```
if (input == "ON") {  
    digitalWrite(13, HIGH);  
    Serial.println("LED is turned ON");  
} else if (input == "OFF") {  
    digitalWrite(13, LOW);  
    Serial.println("LED is turned OFF");  
} else {  
    Serial.println("Invalid input. Type ON or OFF.");  
}  
}  
}
```

Result

- The internal LED blinked at 1-second intervals, and corresponding messages were shown in the Serial Monitor.
- The LED remained continuously ON when programmed accordingly.
- Successfully controlled the LED ON/OFF state dynamically via Serial Monitor using input commands.

Learning Outcomes

- Gained hands-on experience with digitalWrite() for LED control.
- Learned to use the Serial Monitor for output debugging and real-time communication.
- Understood static vs. dynamic output states.
- Explored multiple functional variations using the same hardware.
- Understood basic user-interaction with Arduino via serial input without using external buttons or components.